1 a) in the expression parameters calculated from left to right, the result is as follows.

```
int fun(int *j){
   *j += 5;
   return 2 * (*j) + 1;
}
void main(){
   int x, y, z, v, result1, result2;
   x = 3;
   y = 3;
   z = 3;
   v = 3;
   result1= (x + 2) * fun(&x);
   printf("Result1 = %d\n", sonuc1);
   printf("x = %d\n", x);
  result2 = fun(&y) * (y + 2);
   printf("Result2 = %d\n", sonuc2);
   printf("y = %d\n", y);
   z = z + fun(&z);
   printf("z = %d\n", z);
   v = fun(&v)+ v;
   printf("v = %d\n", v);
}
```

result1 = (3 + 2) * ((2 * 8) + 1)

x=8

Result2 = ((2 * 8) + 1) * (8 + 2)

y=8

z = 3 + ((2 * 8) + 1)

v = ((2 * 8) + 1) + 8

Result1 = 85
x = 8
Result2= 170
y = 8
z = 20
v = 25

b) in the expression parameters calculated from right to left the result is as follows.

```
void main(){

   int x, y, z, v, result1, result2;

   x = 3;

   y = 3;

   z = 3;

   v = 3;

   result1 = (x + 2) * fun(&x);

   printf("Result1 = %d\n", result1);

   printf("x = %d\n", x);

   sonuc2 = fun(&y) * (y + 2);

   printf("Result2 = %d\n", result2);

   printf("y = %d\n", y);

   z = z + fun(&z);

   printf("z = %d\n", z);

   v = fun(&v)+ v;

   printf("v = %d\n", v);

}
```

Result1 = (8 + 2) * ((2 * 8) + 1)

x=8

Result2 = ((2 * 8) + 1) * (3 + 2)

y=8

z = 8 + ((2 * 8) + 1)

v = ((2 * 8) + 1) + 3

Result1 = 170
x = 8
Result2 = 85
y = 8
z = 25
v = 20

c) The results of running the program in Dev C compiler:

Result1 = 170
x = 8
Result2 = 85
y = 8
z = 25
v = 25

d) The results of running the program in Visual Studio:

Result1 = 170
x = 8
Result2 = 85
y = 8
z = 25
v = 25

e) With the same process priority of operant (brackets and function) when used in sequence from left to right are made. As can be seen from the above results; result1 variable screen printing on the first x +2 operator applied operant. Later the value of this operant has changed in fun() function. In the processing for the variable result2, first fun () function is applied to y operant then +2 operator is applied. Also, according to the results obtained in the compiler and also have both z was calculated to be 25. The reason for this, explained as to give priority to the function calls.

2) a) fun () function remains the same and the values of the variables is as follows.

Calling 1:

value=3

list={1,3,5,7,9,11}

Calling 2:

value =3

list={1,3,5,7,9,11}

Calling 3:

value =3

list={1,3,5,7,9,11}

b) fun () function parameters should be sent by reference. Therefore, fun () function and the main function were operated as follows.

```
void fun(int *a, int *b){


  int temp;


  temp = *a;
```

```
  *a = *b;


  *b = temp;


}



void main(){

   int value = 3, list[6] = {1, 3, 5, 7, 9, 11};

   fun(&value, &list[0]);

   fun(&list[0], &list[2]);

   fun(&value, &list[value]);

      }
```

Calling 1:

value =1

list={3,3,5,7,9,11}

calling 2:

value =1

list={5,3,3,7,9,11}

Calling 3:

value =3

list={5,1,3,7,9,11}

c)The changes made in the function are copied to actual parameters. So the results will be identical with the results obtained from pass-by-reference .